

Tab 1

The Vault

David Jackson, Kaila Roberts, Ryan Grimes, Elali McNair, Madison Boyd, Day Ekoi



Design Specifications

Table of Contents

1.0 Introduction (Elali McNair)	
1.1 Goals and objectives	
1.2 Statement of scope	
1.3 Software context	
1.4 Major constraints	
2.0 Data design (Elali McNair)	
2.1 Internal software data structure (Elali McNair)	
2.2 Global data structure (Elali McNair)	
2.3 Temporary data structure	
2.4 Database description (Elali McNair)	
3.0 Architectural and component-level design (Day Ekoi)	
3.1 Program Structure (Day Ekoi)	
3.1.1 Architecture diagram (Day Ekoi)	
3.1.2 Alternatives (Day Ekoi)	
3.2 Description for Web Interface (Day Ekoi)	
3.2.1 Processing narrative (PSPEC) for Web Interface (Day Ekoi)	
3.2.2 Web Interface description (Day Ekoi)	
3.2.3 Sub-Component n.m processing detail	
3.2.3.1 Interface description	
3.2.3.2 Algorithmic model (e.g., PDL)	
3.2.3.3 Restrictions/limitations	
3.2.3.4 Local data structures	
3.2.3.5 Performance issues	
3.2.3.6 Design constraints	
3.3 Description for Flask Application Server (Day Ekoi)	
3.3.1 Processing Narrative (PSPEC) for Flask Application Server	
3.3.2 Flask Application Server Interface Description	
3.3.3 Sub-Component processing detail	
3.3.3.1 Interface description	
3.3.3.2 Algorithmic model (e.g., PDL)	
3.3.3.3 Restrictions/limitations	
3.3.3.4 Local data structures	
3.3.3.5 Performance issues	
3.3.3.6 Design constraints	
3.4 Description for Controller Layer (Day Ekoi)	
3.4.1 Processing Narrative (PSPEC) for Controller Layer	
3.4.2 Controller Layer Interface Description	
3.4.3 Sub-Component processing detail	
3.4.3.1 Create Listing	

3.4.3.2	Algorithmic model (e.g., PDL).....	
3.4.3.3	Restrictions/limitations.....	
3.4.3.4	Local data structures.....	
3.4.3.5	Performance issues.....	
3.4.3.6	Design constraints.....	
3.5	Description for Service Layer (Day Ekoi).....	
3.5.1	Processing Narrative (PSPEC) for Service Layer.....	
3.5.2	Service Layer Interface Description.....	
3.5.3	Sub-Component n.m processing detail.....	
3.5.3.1	Interface description.....	
3.5.3.2	Algorithmic model (e.g., PDL).....	
3.5.3.3	Restrictions/limitations.....	
3.5.3.4	Local data structures.....	
3.5.3.5	Performance issues.....	
3.5.3.6	Design constraints.....	
3.6	Description for Data Model and Database Access Layer(Day Ekoi).....	
3.6.1	Processing Narrative (PSPEC) for Data Model and Database Access Layer.....	
3.6.2	Data Model and Database Access Layer Interface Description.....	
3.6.3	Sub-Component n.m processing detail.....	
3.6.3.1	Interface description.....	
3.6.3.2	Algorithmic model (e.g., PDL).....	
3.6.3.3	Restrictions/limitations.....	
3.6.3.4	Local data structures.....	
3.6.3.5	Performance issues.....	
3.6.3.6	Design constraints.....	
3.7	Software Interface Description (Ryan Grimes).....	
3.7.1	External machine interfaces (Ryan Grimes).....	
3.7.2	External system interfaces (Ryan Grimes).....	
3.7.3	Human interface (Ryan Grimes).....	
4.0	User interface design.....	
4.1	Description of the user interface (Ryan Grimes).....	
4.1.1	Screen images (Ryan Grimes).....	
4.1.2	Objects and actions (Ryan Grimes).....	
4.2	Interface design rules (Ryan Grimes).....	
4.3	Components available (Ryan Grimes).....	
4.4	UIDS description (Ryan Grimes).....	
5.0	Restrictions, limitations, and constraints (Madison Boyd).....	
6.0	Testing Issues.....	
6.1	Classes of tests (David Jackson).....	
6.2	Expected software response (David Jackson).....	
6.3	Performance bounds (David Jackson).....	

6.4 Identification of critical components.....	
7.0 Revision Logs (Madison Boyd).....	

1.0 Introduction (Elali McNair)

Our team, The Vault, is tasked with developing a cloud-hosted campus marketplace that is intended to be used by students enrolled in Hampton University. The system will allow users to log in, browse listings, and post listings using a web-based interface. We chose to develop a fashion marketplace that will allow Hampton University's large fashion community to buy and sell original clothing.

1.1 Goals and objectives

Our goal for our product is to deploy a fully operational cloud-hosted campus marketplace intended for Hampton University students. The system will allow users to securely log in, browse listings, and post simple listings through a web-based interface. We will be using several AWS services to help us create, edit, and deploy our product.

1.2 Statement of scope – Updated by Chi Martin

The software developed for this cloud-based marketplace will allow Hampton University students to be able to buy and sell original clothing. The system will include user authentication, listing management, and database-backed storage through a web-based user interface.

- **Inputs:**
 - User login credentials
 - User-created listing information (title, description, photos, category, etc.)
 - User actions (deleting listings, filters for listings, etc.)
- **Processing functionality:**
 - Authentications and authorization of user using secure password hashing
 - Creation, storage, retrieval, and updates of listings
 - Listing ownership and authorization, ensuring users to edit only the listing they created
 - Cloud-hosted database storage using AWS
- **Outputs:**
 - A catalog of active marketplace listings
 - User-owned listings with options to edit or delete

1.3 Software context

For any and all software being used for our product, the following behavioral requirements are needed:

- **Security**
 - The users login and listing information must be secure at all times
- **Scalability**
 - The marketplace must be able to store a vast amount of users and user listings while the number for both is continuously growing

- Reliability
 - The marketplace must have little to no crashes and cannot to vulnerable to cyber threats
- Usability
 - Users must be able to access the marketplace across macOS, Windows, and multiple web browsers

1.4 Major constraints

The following constraints will affect product development:

- **Technical Constraints:**
 - **Cross-platform compatibility:** The system must function consistently on macOS, Windows, and across web-browsers
 - **Listing ownership:** The system must be developed using AWS which requires compatibility with AWS hosting, networking, and database solutions.
 - **AWS Free tier:** Since the team only has access to AWS Free Tier, the system must be deployed using the free tier, limiting cloud resources and requiring efficient configuration
- **Resource Constraints:**
 - **Time Constraints:** The product has a set timeline that it must be complete within. If it is not completed within the given timeline, it will need to be handed off to the next class to be refined.

2.0 Data Design (Elali McNair) – Updated by Chi Martin

The following internal, global, and temporary data structures are included with our product:

- User login information
- Listing information
- Purchase History
- Wishlist
- Messaging
- Reviews

The user login information will include their Hampton email and password. We want the Hampton email address to be included in this data set in order to keep track of which user the email belongs to when we search through our database either for updates or during the verification process.

The listing information will include the username, the title of the listing, any images of the listing, the amount for sale, and the price. The information listed after the username is strictly for the details of the listing itself. The username is included to keep track of what user the listing belongs to and keep track of how many listings the user has created.

The purchase history data set will include the username, the titles of the listings the user has purchased, any images of the listing, the amount for sale, and the price. This is to try and keep an accurate track of what the user has bought and how many they have purchased.

The wishlist data set will include the username, the titles of the listings the user has marked, any images of the listing, the amount for sale, and the price of the listing. Almost like a shopping list, this is to work as a future purchase feature for the user to remember what they would like to purchase. To make this feature operational and accurate we need to have nearly all of the listing information included, similar to how it is listed in the purchase history data set and the listing information data set.

The messaging data set will include the sender, receiver, listing, message text, time the message was read, and creation time. Similar to most shopping apps, this feature will allow for users to message one another regarding listings. It also will allow users to see the exact time and date the message was sent/received.

The reviews data set will include the storefront, reviewer, rating, and comments. It will give users, including admins, the ability to review a listing and comment on its overall quality. On the other hand, it allows the owners of listings to see various reviews/comments made on their post.

2.1 Internal software data structure (Elali McNair)

The data structures that are passed through small portions of the product are the user information. Since we need to make sure that their profile is updated across all profile pages in the product anytime the user information is changed or added it will be passed to the database for storage then updated in their profile page for them and other users to see, depending on what information is updated.

2.2 Global data structure (Elali McNair) – Updated by Chi Martin

The data structure that would be prevalent throughout the product will be the listing information. This is because the listing information is used for creating and updating the user's wishlist, purchase history, and checkout cart. Even though the user's wishlist, purchase history, and checkout cart are not public information for every user to view, the listing information being saved to each of these sections is public information within the product. The data structure for messaging allows for communication between different users. There is a sender, receiver, a listing, a status for when a message was read, and a creation date. The reviews data structure is for user feedback on listings made on the storefront. Each review is directly associated with a storefront, reviewer, rating, comment, and timestamp for when it was made.

2.3 Temporary data structure

Files created for interim use are described.

2.4 Database description (Elali McNair) – Updated by Chi Martin

The following information is stored into The Vault database:

- User login information
 - Username (ex. Hampton email)
 - Password
 - Role (regular user or admin)

The database to store the login information of created users has been made using AWS RDS and PostgreSQL. It stores usernames and passwords for regular users and stores special keys belonging to the admin user of the database. These keys have been generated and connected to the database using AWS Endpoint and AWS IAM.

- Listings
 - Title
 - Images
 - Fulfillment type
 - Count
 - Price
 - Inventory
 - Status

The database has been edited to store the listing information created by any user. Due to the “Query Editor” being taken off of the AWS RDS service, we will have to either use PostgreSQL for the updates or another service. This set of data needs to be constantly secured and updated since it will also be used for several other lists within the marketplace.

- Listing sizes

This table maintains the size-based inventory quantities for listings

- Listing images

This table stores S3 image URLs associated with listings.

- Purchase History
 - User
 - Listing title
 - Listing image
 - Count
 - Price

The database has been updated to store the purchase history of every user on the marketplace. The information within this data set will be nearly the same as the listing information with the only difference being the inclusion of the user. This is so the user can be able to view the proper amount of information once the feature is operational.

- Wishlist
 - User
 - Listing title
 - Listing image
 - Count
 - Price

The database has been updated to store the wishlist of every user on the marketplace. The information within this data set is exactly the same as the purchase history data set. This is because the wishlist format will be the same as the purchase history format and to make the implementation process easier and display the proper amount of information to the user, the data remains the same.

- Storefronts
 - Brand name
 - Bio
 - Logo
 - Banner
 - Contact information
 - Preview Images
 - Categories

The database has been updated to store the storefronts of every user on the marketplace. This table contains as much information as the listings table due to the amount of information the user needs to enter to make their storefront appealing to other users. The information also allows other users to easily make purchases from that storefront and make the transaction as simple as possible.

- Cart items
 - Listings
 - Price
 - Count

The database has been edited to store the cart items of the user.

- Orders
 - Buyer details
 - Meeting location
 - Confirmation number

The database has been updated to store the orders every user has made. This is to keep track of what they have purchased and confirm to the sellers that their items have been bought.

- Order items

The database has been updated to store the individual items associated with each order.

- Returns
 - Order number
 - Reason
 - Damage status
 - Optional S3 damage image URL

The database has been updated to store the return requests made by every user in the marketplace. This is to confirm that any return request is validated and properly carried out.

- Messages
 - Sender
 - Receiver
 - Listing
 - Body
 - Is Read
 - Created At

We plan to update the database to include messaging between users of the site.

- Reviews
 - Storefront
 - Reviewer
 - Rating
 - Comment
 - Created At

The database also stores reviews made by different users/admins. We are including this feature to ensure that users will be able to see feedback from their fellow users as well as the site admins.

3.0 Architectural and component-level design (Day Ekoi)

This section describes the architectural structure of The Vault system and provides a detailed overview of the software components that make up the application. The architecture defines how the system is organized, how different components interact, and how responsibilities are distributed throughout the application.

The Vault follows a modular backend architecture built using the Flask web framework. The system separates responsibilities into multiple logical layers such as controllers, services,

models, and database layers. This layered design improves readability, maintainability, and scalability.

The application is designed as a web-based system where users interact with the platform through a browser interface. Requests are sent to the backend server, processed through application components, and data is stored or retrieved from AWS RDS. Additionally, the architecture also integrates external cloud services such as Amazon S3 for image storage.

3.1 Program Structure (Day Ekoi) – Updated by Chi Martin

As previously iterated, The Vault uses a layered program structure that separates responsibilities across multiple components. This structure ensures that each layer of the system handles a specific function and interacts with other layers in a controlled manner.

The primary layers of the application include:

User Interface Layer

- This layer provides the web-based interface used by shoppers, brand owners, and administrators.
- Displays storefronts, listings, and user actions such as browsing, purchasing, or managing storefronts and listings.

Authentication Layer

- Handles user registration, login, logout, and session management via Flask sessions.
- Stores user identity, role, and cart state in the session on login.
- Enforces Hampton University email validation during registration using werkzeug password hashing
- Implemented in `auth_controller.py`, `auth_services.py`, and `auth_utils.py`

Application Layer (Flask API)

- Handles incoming HTTP requests from the frontend interface
- Routes requests to the appropriate controller based on the endpoint being accessed.
- Hosted and served through AWS Elastic Beanstalk, which manages the deployment environment, scaling, and HTTPS request handling.

Controller Layer

- Defines API routes and receives client requests using Flask Blueprints.
- Parses request data and forwards it to the appropriate service layer functions.
- Covers storefront management, listing management, cart, checkout, orders, wishlist, returns, admin operations, messaging, and reviews.

Service Layer

- Handles the system rules, validations, and permission checks before interacting with the database

- Enforces ownership rules, role-based access control, and input validation across all operations.

Model Layer

- Handles communication with the database.
- Executes SQL queries via psycopg2 and returns structured data to the service layer.

Database Layer

- Stores all persistent system data including users, storefronts, listings, orders, cart items, returns, messages, reviews, and related entities across [insert actual number] tables.
- Hosted using AWS RDS with PostgreSQL.
- Network access to the database is controlled through AWS Security Groups, which restrict inbound connections to the Elastic Beanstalk environment only, communicating over TCP/IP on port 5432.
- AWS IAM manages access permissions across the database and cloud infrastructure.
- Amazon VPC provides network-level isolation between the application server and database layer.

External Storage Layer

- Handles storage of all image files using Amazon S3 via boto3, implemented in utils/s3.py.
- Used for storefront banner images, logos, and up to four preview images per storefront; up to four listing images per listing; and damage images submitted through the returns system.
- Image files are uploaded to the S3 bucket organized by folder (storefronts/, listings/, returns/damage/) with unique UUID-generated filenames.
- Image public URLs are stored as references in the database and rendered directly in the browser.
- AWS IAM controls S3 access permissions and bucket policy manages public read access for image rendering.

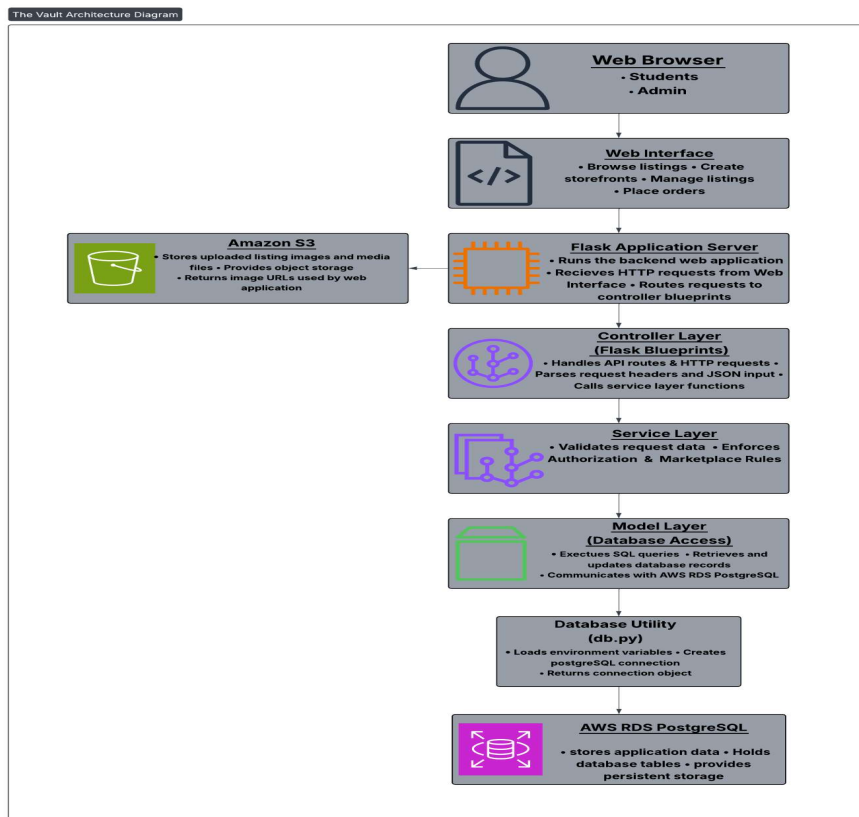
This layered architecture ensures that responsibilities are clearly separated and allows the system to remain modular, easier to read, and maintain.

3.1.1 Architecture diagram (Day Ekoi)

The architecture of The Vault is represented through a layered diagram that illustrates how system components interact with each other.

User interactions first originate from a web browser interface. Requests are then sent to the backend Flask server where controllers handle routing and request parsing. Controllers call service layer functions to apply rules and validation. The service layer interacts with the model layer to execute database operations. The database layer stores and retrieves information from AWS RDS. Listing images are stored externally in Amazon S3 and referenced within the database.

Architecture Flow:



This structure ensures that the system can support scalable data management and integrate securely with external cloud infrastructure.

3.1.2 Alternatives (Day Ekoi)

Several alternate architectural approaches were considered during the design of The Vault. While from the start it was known it would not be the most useful architecture, a monolithic architecture was an option where all system components operated within a single application. It was chosen because it simplifies development, however it was quickly realized by the scope of this project that it can make the system harder to read, maneuver through, maintain, and scale.

Another option was a serverless architecture using AWS services such as Lambda were also considered. This approach can improve scalability and reduce infrastructure management.

For this project, a layered architecture using a Flask-based backend was selected and fully implemented. The system separates responsibilities across the user interface, authentication, controller, service, model, and database layers. The application is deployed on AWS Elastic Beanstalk, which handles server management, environment configuration, and HTTPS request

handling, allowing the team to focus on application development rather than infrastructure. This layered structure backed by AWS infrastructure improves organization, maintainability, and development efficiency across all five iterations of the project.

3.2 Description for Web Interface (Day Ekoi)

The Web Interface component represents the user interface portion of The Vault system. This layer allows students and admins to interact with the marketplace through a web browser. The interface is responsible for displaying listings, storefronts, and account information while capturing user input such as login credentials, listing creation data, and purchase actions.

The interface communicates with the backend application through HTTP requests sent to the Flask API endpoints. Responses returned from the backend are then rendered to users in the form of listings, account information, or confirmation messages.

3.2.1 Processing narrative (PSPEC) for Web Interface (Day Ekoi)

The Web Interface processes user actions and forwards them to the backend server. When a user performs an action such as browsing listings, creating a storefront, or placing an order, the interface sends a request to the correct backend API endpoint.

The backend processes the request, retrieves or modifies the necessary data, and returns a response that is displayed to the user through the interface.

3.2.2 Web Interface description (Day Ekoi)

The inputs and outputs associated with the Web Interface component are described below.

Inputs to the Web Interface:

- User credentials for authentication (email, username, password)
- Storefront information such as brand name, logo, description, and banner and logo image files
- Listing information entered by brand owners (title, description, price, fulfillment type, and inventory details)
- Listing image files submitted during listing creation, which are uploaded to Amazon S3 via the backend and referenced within the database
- User navigation requests when browsing storefronts and listings
- Search input used to filter and retrieve storefronts in real time
- Order and checkout actions initiated by shoppers, including buyer contact details and meeting location
- Wishlist selections and cart management actions taken by users interacting with marketplace items
- Return request submissions including order number, reason, and optional damage image

These inputs are captured through form submissions, buttons, and user interface actions within the browser.

Outputs generated by the Web Interface:

- Displays marketplace storefronts and connected listings information retrieved from the backend
- Real-time search results filtering storefronts based on user input
- Confirmation messages when actions such as storefront or listing creations are successful
- Order confirmation page displaying confirmation number, order summary, and total amount
- Error messages when invalid input or unauthorized actions occur
- Updated views of storefronts, listings, cart contents, orders, or wishlist items based on backend responses

The Web Interface sends requests to the Flask backend using standard HTTP methods such as GET, POST, DELETE, and receives responses in JSON format, which are then rendered into visible content within the browser. Overall, the interface allows users to interact with the marketplace system while the backend application handles processing, validation, and data management.

3.2.3 Sub-Component: S3 Image Upload

A detailed algorithmic description of the image upload sub-component within the Web Interface is presented below. This sub-component handles all image file submissions from the user interface to AWS S3 storage and applies across storefront creation, listing creation, storefront editing, listing editing, and return submissions.

3.2.3.1 Interface description

The image upload sub-component accepts image files submitted by users through the web interface via drag-and-drop zones or file browser inputs. Files are submitted as multipart/form-data to the appropriate backend endpoint. The sub-component outputs a public S3 URL that is stored in the database and rendered back to the user in the interface.

3.2.3.2 Algorithmic model (e.g., PDL)

1. User selects or drags an image file into the upload zone in the browser
2. JavaScript captures the file and submits it to the appropriate backend endpoint as multipart/form-data
3. Flask receives the file via request.files and validates the file extension
4. utils/s3.py generates a unique filename using UUID and uploads the file to the S3 bucket under the appropriate folder (storefronts/, listings/, or returns/damage/)
5. The public S3 URL is constructed and returned to the controller as a JSON response
6. The URL is stored in the relevant database table and rendered in the interface

3.2.3.3 Restrictions/limitations

- Requires a valid AWS S3 bucket configured with public read access via bucket policy
- Requires AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION, and AWS_S3_BUCKET environment variables to be set in the application environment
- Accepted file types: PNG, JPG, JPEG, GIF, WebP only. All other file types are rejected before upload
- Maximum 4 images per listing; 1 banner, 1 logo, and 4 preview images per storefront; 1 damage image per return submission
- Requires a stable network connection between the Flask server hosted on AWS Elastic Beanstalk and the AWS S3 service

3.2.3.4 Local data structures

Image files are handled as Flask file objects retrieved from request.files. After a successful upload, the returned S3 URL is stored as a TEXT value in the appropriate database table: listing_images for listing images, storefronts for banner, logo, and preview images, and returns for damaged images. Filenames are generated at runtime using uuid.uuid4().hex combined with the original file extension to ensure uniqueness.

3.2.3.5 Performance issues

Large image files or multiple simultaneous uploads may increase response times. Each upload requires a round-trip network request from the Elastic Beanstalk server to AWS S3. Upload failures return None from upload_image_to_s3() and are handled with structured error responses returned to the client. Under the AWS Free Tier, bandwidth and request limits may impact upload performance during periods of high activity.

3.2.3.6 Design constraints

The sub-component is constrained to AWS S3 as the image storage provider, consistent with the overall AWS infrastructure used by the project. Image URLs must be publicly accessible since they are rendered directly in the browser without additional authentication. The upload logic is centralized in utils/s3.py to keep it reusable across the storefront, listing, and return contexts. The sub-component must also remain compatible with the multipart/form-data encoding required for file submissions through HTML forms and JavaScript fetch requests.

3.3 Description for Flask Application Server (Day Ekoi)

The Flask Application Server serves as the central processing layer of The Vault system. It receives incoming HTTP requests from the Web Interface and routes those requests to the appropriate controllers for processing.

The server coordinates communication between the user interface, backend application components, database system, and external storage services.

3.3.1 Processing Narrative (PSPEC) for Flask Application Server

When a request is received from the Web Interface, the Flask server determines the appropriate API route and forwards the request to the relevant controller. The controller then processes the request and communicates with other application layers to complete the requested operation.

Once processing is finished, the Flask server sends a structured response back to the Web Interface, allowing the user interface to update the display as needed.

3.3.2 Flask Application Server Interface Description

The Application Server receives HTTP requests from the Web Interface and communicates with backend components to process those requests.

Inputs received by the Flask Server include:

- HTTP requests sent from the Web Interface
- Request headers containing user authentication information
- Request payloads containing data related to storefronts, listings, or orders.

Outputs produced include:

- JSON responses containing requested data
- Confirmation responses for successful operations
- Error responses for invalid or unauthorized requests

The Flask server forwards requests to the controller layer, which then interacts with the service and data layers of the application.

3.3.3 Sub-Component processing detail

The Flask Application server has several sub components that handle request routing, application processing, and database interaction. These sub-components include the controller layer, service layer, and model (data access) layer.

To break down further, when an HTTP request is received, the Flask routing system directs the request to the appropriate controller. The controller validates the request and calls functions in the service layer responsible for performing the requested operation. The service layer is responsible for system functionality and interacts with the model layer to retrieve or store data in the PostgreSQL database.

Once completed, the results or status information is returned to the controller, which formats the response as a JSON object and sends it back through the Flask server to the Web Interface. This layered structure separates responsibilities across components and allows each sub-component to perform a clearly defined role in processing application requests.

3.3.3.1 Interface description

Each sub-component within the Flask Application Server communicated with other components through clearly defined function calls and data structures.

Inputs into sub-components include:

- HTTP request data sent from the Flask routing system
- Request parameters such as storefront IDs, listing IDs, and user identifiers
- JSON payload data containing listing, storefront, or order information

Outputs produced:

- Structured data objects retrieved from the PostgreSQL database
- JSON formatted responses returned from the controller layer
- Status messages indicating successful operations or errors

Controllers serve as the primary interface between the Flask server and internal processing layers, while the service and model layers provide support for system operations.

3.3.3.2 Algorithmic model (e.g., PDL)

The following pseudocode:

```
BEGIN Flask_Server_Request_Handler
```

```
    Receive HTTP request from Web Interface
```

```
    Determine requested API route
```

```
    IF route exists THEN
```

```
        Forward request to corresponding controller
```

```
        Controller processes request
```

```
        Controller calls service layer if business rules are required
```

```
        Service layer interacts with model layer for database operations
```

```
        Receive response from controller
```

```
        Return JSON response with appropriate HTTP status code
```

```
    ELSE
```

```
        Return HTTP 404 Not Found
```

```
    END IF
```

```
END
```

3.3.3.3 Restrictions/limitations

The Flask server depends on several external system components and infrastructure services in order to function correctly.

Key elements include:

- A running PostgreSQL database instance hosted on AWS RDS for persistent data storage
- Valid environment configuration variables for database credentials and service endpoints
- A stable network connection between the Flask Server and the database system
- The presence of required Python libraries and application dependencies

If any of these components are unavailable, certain system operations such as data retrieval, listing creation, or storefront management may fail or return errors.

3.3.3.4 Local data structures

The Flask server does not actually maintain internal data structures. It instead primarily handles structured request and response objects provided by the Flask framework. Incoming requests are represented using Flask request objects that contain headers, URL parameters, and JSON payloads. Responses are returned as JSON objects using Flask's `jsonify()` utility. These structures allow the application server to pass data between the controller, server, and model layers.

3.3.3.5 Performance issues

Potential performance issues include high volumes of simultaneous HTTP requests, repeated queries, and image retrieval from cloud storage. Inefficient queries or large requests could increase response times. Additionally, some other aspects of the marketplace that may affect the run-time performance, security, or computational accuracy would be the following:

- User personal information storage
- Banking or card detail storage
- “Non-student” account registration

Storing the user's personal information, bank or card details, or any non-student account emails can compromise the security of the marketplace or the users within it. Since this marketplace is specifically for students enrolled in Hampton University, the marketplace cannot collect any information on its users other than their email address to verify if they're enrolled in the university. By collecting and storing their personal and banking information, The Vault would be phishing for unimportant information. If The Vault tries to store the email address of a user who is not enrolled in the university, then that user now has access to a campus-private marketplace and can potentially attack or steal any collected information from other users on the marketplace.

3.3.3.6 Design constraints

The following design constraints apply to the Flask Application Server and the overall system architecture:

- **AWS Infrastructure dependency** - the application is deployed on AWS Elastic Beanstalk and depends on AWS RDS for database management, Amazon S3 for image storage, AWS IAM for access control, AWS Security Groups for network-level firewall rules, and Amazon VPC for infrastructure isolation. The system must remain compatible with all of these services.
- **AWS Free Tier limitations** - the team operates within AWS Free Tier limits, constraining available compute resources, database storage, memory allocation, and concurrent connections. All architectural decisions must account for these resource boundaries.
- **Non-student account registration** - the system enforces Hampton University email validation at the service layer. Only emails ending in hamptonu.edu or my.hamptonu.edu are permitted to register, restricting marketplace access to verified Hampton University students.
- **Cross-platform compatibility** - the system must function consistently across macOS and Windows and across major web browsers. The frontend is built with standard HTML, CSS, and JavaScript to ensure broad compatibility without reliance on browser-specific features.
- **One storefront per user** - the service layer enforces a rule that each user account may only create one storefront. This constraint is validated in storefront_service.py before any database insertion occurs.
- **In-person exchange model** - the checkout system collects buyer contact information and a meeting location rather than processing payments. Sensitive payment data is never collected or stored in the database. This is a deliberate design constraint for the current iteration of the product.
- **Layered architecture enforcement** - controllers must not access the database directly. All database operations must flow through the service and model layers to maintain separation of concerns and preserve the integrity of the system architecture.

3.4 Description for Controller Layer (Day Ekoi)

The Controller Layer defines the API endpoints for The Vault using Flask Blueprints. It acts as a gateway between incoming HTTP requests and the backend functions. Controllers interpret requests, extracting needed values (URL parameters, request headers, and JSON requests), and forward the inputs to the correct service layer functions. After the service layer is done processing, controllers return JSON responses and apply appropriate HTTP status codes.

Currently, controllers use a temporary authentication approach by reading information from request headers. This provides a temporary working mechanism, to test authorization before full login/session authentication is connected.

3.4.1 Processing Narrative (PSPEC) for Controller Layer

When a request arrives at the Flask server, the controller blueprint matches the request to a route based on the HTTP method and URL path.

For page routes and session-protected endpoints, the controller checks the Flask session for the presence of session['user'], session['user_id'], and session['role'] to verify that the user is authenticated. If the session is missing or invalid, the user is redirected to the login page. This is the primary authentication mechanism used throughout the application, established when a user successfully logs in through auth_controller.py.

For direct API endpoints, the controller uses a secondary helper function get_current_user() that reads X-User-Id and X-User-Role request headers to identify the requesting user. This approach supports direct API access and testing outside of the browser session context.

The controller parses incoming request data using request.get_json(silent=True) for JSON payloads or request.form for multipart form submissions including file uploads. The controller then calls the corresponding service function, passing the current user, any route parameters such as storefront_id or listing_id, and the parsed input data. The service layer performs all rule enforcement and database operations through the model functions.

A successful service call returns the result as JSON using jsonify() along with the correct HTTP success status code. If an error occurs, the controller catches the exception and returns a structured JSON error message. Errors are mapped to status codes based on the type of failure: 401 for unauthorized, 403 for forbidden, 404 for missing resources, and 400 for invalid input.

3.4.2 Controller Layer Interface Description

The Inputs and outputs received by controllers include:

Inputs:

- HTTP requests routed to the appropriate blueprint (POST, GET, PUT, PATCH, DELETE)
- URL parameters such as storefront_id, listing_id, user_id, and return_id
- JSON request bodies containing data for create and update actions
- Multipart form data containing image file uploads for listings, storefronts, and returns
- Flask session data containing user identity (session['user'], session['user_id'], session['role'])
- Request headers containing X-User-Id and X-User-Role for direct API access

Outputs:

- Structured JSON responses returned to the Flask server
- Error responses when validation or permission checks fail
- HTTP status codes that indicate success or failure
- Page redirects when session authentication fails on protected routes

3.4.3 Sub-Component processing detail

The following sub-components represent key controller endpoints implemented for functionality.

3.4.3.1 Create Listing

Inputs:

- HTTP Method: POST
- Route: /api/listings/create
- Session: session['user_id'] (required)
- Form Data (multipart/form-data):
 - title, price, fulfillment_type, status
 - sizes_available (JSON string array)
 - size_quantities (JSON object mapping size to quantity)
 - quantity_on_hand (calculated from size quantities)
 - is_made_to_order (boolean)
 - listing_images (up to 4 image files, uploaded to S3)

Outputs:

- HTTP 201 + JSON listing record on success
- HTTP 401 if no valid user header exists
- HTTP 403 if user lacks permission
- HTTP 404 if storefront does not exist
- HTTP 400 for validation errors
- HTTP 500 if S3 image upload fails

3.4.3.2 Algorithmic model (e.g., PDL)

The following pseudocode represents the logic used by the controller when handling a listing creation request. The pseudocode reflects behavior implemented in listing_controller.py, storefront_controller.py, and all other files in the controller folder where the controller validates the requesting user, parses request data, calls the service layer, and returns the resulting JSON response.

```
FUNCTION create_listing_from_form():  
  
    current_user = get_current_user_from_session()  
    IF current_user is None:  
        RETURN 401 Unauthorized  
  
    user_storefront = get_my_storefront_service(current_user)  
    IF user_storefront is None:  
        RETURN 400 "You must create a storefront before creating listings"  
  
    PARSE form fields: title, price, fulfillment_type, status,  
                      sizes_available, size_quantities, is_made_to_order  
  
    IF title is empty:  
        RETURN 400 "title is required"
```

```
IF sizes_available is empty:
    RETURN 400 "At least one size must be selected"

IF no image files submitted:
    RETURN 400 "At least one listing image is required"

IF more than 4 image files submitted:
    RETURN 400 "You can upload up to 4 listing images"

FOR each image file:
    IF invalid file type:
        RETURN 400 "Invalid file type"
    image_url = upload_image_to_s3(file, folder="listings")
    IF upload fails:
        RETURN 500 "Failed to upload image to S3"
    APPEND image_url to image_urls

quantity_on_hand = SUM of all size quantities

listing = create_listing_service(current_user, storefront_id, data)

FOR each image_url in image_urls:
    add_listing_image_service(current_user, listing_id, image_url, is_primary=(index == 0))

FOR each size in sizes_available:
    upsert_listing_size_service(current_user, listing_id, size, size_quantity)

RETURN 201 + JSON listing record
```

3.4.3.3 Restrictions/limitations

The Create Listing controller endpoint requires a valid authenticated session. If the user is not logged in, the request cannot continue.

The user must also have an existing storefront before creating a listing. This endpoint depends on the service layer to enforce permission rules such as storefront ownership and administrator privileges. Successful execution additionally requires the database connection, model layer, and AWS S3 service to be available to complete the required operations.

3.4.3.4 Local data structures

The controller layer uses structured request and response objects. For the Create Listing endpoint, request data is parsed from multipart form data using `request.form` and `request.files`. This allows the controller to access fields such as `title`, `price`, `fulfillment_type`, `sizes_available`, `size_quantities`, and `is_made_to_order`. Image files are retrieved from `request.files` as Flask file objects and passed to `utils/s3.py` for upload.

User identity is retrieved from the Flask session using `session['user_id']` and passed to service functions as a `current_user` dictionary containing id and role. These structures are passed to the service layer for validation and processing.

3.4.3.5 Performance issues

Possible performance issues for the controller layer include the number of incoming HTTP requests and the size of the requests. If a large number of requests are received at the same time, the server may experience increased response times. For the Create Listing endpoint specifically, uploading multiple image files to AWS S3 within a single request adds additional processing time as each file requires a separate round-trip upload operation. Large image files or slow network conditions between the Elastic Beanstalk server and S3 could further affect response times. Proper request validation, efficient routing, and minimization of unnecessary data processing help ensure the controller layer remains responsive.

3.4.3.6 Design constraints

The main design constraint is that the controller layer must follow the layered architecture created for the system, therefore it cannot directly access the database. All database operations must occur through the service and model layers. Controllers are simply responsible for request parsing, routing, and returning structured responses. Additionally, it must maintain compatibility with Flask and support REST-style API endpoints using HTTP methods. Additionally, file upload handling must remain compatible with multipart/form-data encoding and AWS S3 via `utils/s3.py`, and session-based authentication must be checked on all protected routes before any processing occurs.

3.5 Description for Service Layer (Day Ekoi)

The Service Layer is responsible for applying system rules, validating user input, and enforcing permissions within the application. This component ensures that operations performed by users follow the systems defined rules before database actions occur. The files in this layer confirm that the requesting user is authorized to perform an action, validate required input fields, enforce marketplace constraints (such as fulfillment type rules), and then call model-layer functions to execute the corresponding database operations.

The Vault's service layer covers storefront, listing, authentication, checkout, wishlist, purchase, returns, and admin operations, implemented across `storefront_service.py`, `listing_service.py`, `auth_services.py`, `checkout_service.py`, `wishlist_service.py`, `purchase_service.py`, `return_service.py`, and `admin_service.py`. Centralizing these operations ensures that all actions apply consistent permission checks and validation behavior.

3.5.1 Processing Narrative (PSPEC) for Service Layer

More specifically, when a controller calls a listing service function, the service first verifies that a valid `current_user` exists. The service then checks and confirms that the relevant resource exists, such as verifying the storefront exists before creating a listing, or verifying that the listing

exists before updating images or size inventory. After confirmation that it exists, the service enforces authorization rules by ensuring the requesting user is a storefront owner associated with the listing or an admin.

Next, the service validates request data to ensure required fields are present, correctly typed, and within accepted constraints. For listings, this includes validating title, price, fulfillment type and inventory values, as these are required. For listing images, the service validates that an image URL is provided and enforces rules such as only allowing the owner or admin to set a primary image. For size inventory, the service verifies size input and ensures quantity values are positive integers. For checkout, the service validates that all buyer details are present, calculates the order total, generates a confirmation number, creates the order and order item records, decrements size inventory, clears the cart, and removes purchased items from the user's wishlist.

After successful validation and authorization, the service calls model functions to execute database queries. The model returns structured records, which the service returns back to the controller.

3.5.2 Service Layer Interface Description

The inputs received by the service layer include:

- `Current_user` object containing user id and role
- Resource identifiers such as `storefront_id`, `listing_id`, `image_id`, `return_id`, and `order_id`
- Processed request data forwarded from controllers including listing attributes, storefront fields, buyer details, and cart contents
- User identification information used for permission validation
- Image URLs returned from S3 uploads passed through from the controller layer
- Session cart data passed to the checkout service for order processing

Outputs produced by the service layer include:

- Created or updated records returned as dictionaries
- Lists or records for retrieval functions
- Order records including confirmation number, order items, and total amount
- Errors when requests are invalid, unauthorized, or missing.

3.5.3 Sub-Component n.m processing detail

The following sub-components represent key service operations implemented in `listing_service.py`. But, these operations are implemented similarly in practically every service layer file including storefront, checkout, wishlist, returns, and admin services. The listing service functions manage all listing related operations within the marketplace. IT handles actions such as creating and updating listings, adding images to listings, and managing size based inventory records. Each operation begins by verifying that the user is authenticated and authorized to

modify the resource. The service confirms that the listing and its associated storefront exist and ensures that the requesting user is either the storefront owner or an admin. After authorization checks, the service validates input data to confirm required fields are present and that values are correctly formatted. This includes validating listing attributes such as title, price, fulfillment type, description, quantity, image URLs. Once validation is complete, the service calls the appropriate model-layer function to execute the database operation. The resulting record is then returned to the controller layer, which fronts the response for the client.

3.5.3.1 Interface description

The service layer receives validated request data and user information from the controller layer. Inputs include the `current_user` object, identifiers such as `storefront_id` or `listing_id`, and request data containing listing attributes, image URLs, size inventory values, or buyer details depending on the operation. Outputs from the service layer include structured records representing created or updated data, lists of retrieved objects, or error responses when operations fail.

3.5.3.2 Algorithmic model (e.g., PDL)

```
FUNCTION create_listing_service(current_user, storefront_id, data):  
  
    IF current_user is None:  
        RAISE "Unauthorized"  
  
    storefront = get_storefront_by_id(storefront_id)  
    IF storefront is None:  
        RAISE "Storefront not found"  
  
    IF storefront.owner_id != current_user.id AND current_user.role != "admin":  
        RAISE "Unauthorized action"  
  
    VALIDATE title, price, fulfillment_type from data  
    IF fulfillment_type == "IN_STOCK":  
        VALIDATE quantity_on_hand >= 0  
    IF fulfillment_type == "PREORDER":  
        SET quantity_on_hand = None  
  
    listing = create_listing(storefront_id, validated data)  
    RETURN listing
```

3.5.3.3 Restrictions/limitations

Service layer operations require a valid authenticated user and an existing resource record such as a listing or storefront. The database must be reachable and properly initialized for operations to succeed. For checkout service operations, the session cart must be non-empty and all buyer details must be present before an order can be created.

3.5.3.4 Local data structures

The service layer primarily uses dictionary-style data structures for representing user information (current_user) and request data (data). These structures contain listing attributes, image URLs, or size inventory values depending on the operation being performed. The checkout service additionally uses a list of session cart item dictionaries to calculate order totals and create order item records.

3.5.3.5 Performance issues

Service layer performance can be affected by the number of sequential database calls required to complete a single operation. For example, creating a listing with multiple images and sizes requires separate model calls for the listing record, each image, and each size entry. The checkout service performs multiple sequential operations including creating the order, creating order items, decrementing size inventory, clearing the cart, and removing purchased items from the wishlist. These operations are executed in sequence and could be affected by database response times under load.

3.5.3.6 Design constraints

The service layer must enforce authorization rules consistently across all listing operations and must not allow controllers to directly access the database. All database interactions must occur through model-layer functions to preserve the layered architecture of the system. Additionally, The service layer must also rely on the model layer for all SQL execution and connection handling.

3.6 Description for Data Model and Database Access Layer(Day Ekoi)

The Data Model Layer manages communication between the application and the database. This layer is responsible for executing SQL queries and retrieving or modifying records within the database.

3.6.1 Processing Narrative (PSPEC) for Data Model and Database Access Layer

This layer is important for when the service layer requires data retrieval or modification. It sends requests to the data model layer and it executes the necessary SQL queries and returns the resulting records to the service layer.

3.6.2 Data Model and Database Access Layer Interface Description

The inputs received by the model layer include:

- Data requests forwarded from the service layer
- Query parameters used to identify database records
- Structured data representing new or updated records to be inserted or modified in the database

Outputs produced by the model layer include:

- Database query results returned as structured data in dictionary format
- Confirmation of successful database insertions, updates, or deletions
- Error messages if database operations fail

The model layer communicates directly with the PostgreSQL database hosted on AWS RDS using psycopg2 for all SQL execution and connection handling.

3.6.3 Sub-Component n.m processing detail

The Data Model and Database Access Layer is composed of multiple model modules responsible for interacting with specific database tables. These sub-components correspond to core system entities such as users, storefronts, listings, listing sizes, listing images, purchases, wishlists, cart items, orders, order items, and returns.

Each model module contains functions that execute SQL queries to retrieve, insert, update, or delete records within the PostgreSQL database. These functions are called by the service layer when application operations require database interaction.

When a request is received from the service layer, the appropriate model function constructs and executes the required SQL query using the database connection. The results of the query are then formatted into structured data objects and returned to the service layer for further processing.

This design isolates database interaction within the model layer, ensuring that application logic remains separate from direct database operations.

3.6.3.1 Interface description – Updated by Chi Martin

The model sub-components provide a programmatic interface for accessing and modifying database records.

Inputs include:

- Data requests forwarded from the service layer
- Query parameters such as user IDs, storefront IDs, listing IDs, order IDs, return IDs, sender IDs, receiver IDs, reviewer IDs, and confirmation numbers
- Structured data representing new or updated database records

Outputs include:

- Database query results returned as structured data objects
- Confirmation responses indicating successful record creation or modification
- Error information when database operations fail or return invalid results

Model functions act as the primary interface between the application and the PostgreSQL database system.

3.6.3.2 Algorithmic model (e.g., PDL)

```
FUNCTION create_listing(storefront_id, title, price,
    fulfillment_type,
        quantity_on_hand, status, description,
    is_made_to_order):

    conn = get_connection()
    cur = conn.cursor()

    EXECUTE SQL:
        INSERT INTO listings (storefront_id, title, price,
            fulfillment_type,
                quantity_on_hand, status, description,
            is_made_to_order)
            VALUES (storefront_id, title, price, fulfillment_type,
                quantity_on_hand, status, description,
            is_made_to_order)
            RETURNING all fields

    result = cur.fetchone()
    conn.commit()
    cur.close()
    conn.close()

    RETURN result as dictionary
```

3.6.3.3 Restrictions/limitations

The Data Model and Database Access Layer depends on several external components in order to function properly.

Key requirements include:

- A functioning PostgreSQL database instance hosted on AWS RDS
- Valid database credentials and environment variables configured within the Elastic Beanstalk application environment
- A reliable network connection between the Elastic Beanstalk application server and the AWS RDS database instance, controlled through AWS Security Groups which restrict inbound database access to port 5432

- Proper database schema initialization via `init_db.py` including the required tables and relational constraints

If these requirements are not met, database operations may fail, resulting in error responses being returned to the service layer.

3.6.3.4 Local data structures – Updated by Chi Martin

The Data Model Layer operates primarily on relational data structures defined within the PostgreSQL database schema. These structures include tables representing the core entities of the system.

Key database tables include:

- **users** - stores user account information including username, hashed password, email, and role
- **storefronts** - stores storefront ownership and configuration data including brand name, bio, logo, banner, contact info, preview images, and categories
- **listings** - stores product listing information including title, price, fulfillment type, inventory, and status
- **listing_sizes** - maintains size-based inventory quantities for listings
- **listing_images** - stores S3 image URLs associated with listings
- **purchases** - records completed transactions
- **wishlist** - stores user-saved listings
- **cart_items** - stores persistent shopping cart items per user
- **orders** - stores completed checkout orders including buyer details, meeting location, and confirmation number
- **order_items** - stores individual line items associated with each order
- **returns** - stores return requests including order number, reason, damage status, and optional S3 damage image URL
- **messages** – stores text sent between users, including a sender, receiver, the relevant listing, message text, read status, and time of creation.
- **reviews** – stores reviews made of listings, including the actual storefront, reviewer, the rating, comments, and the time of creation.

These relational tables are connected through primary and foreign key relationships to maintain data integrity across the system.

3.6.3.5 Performance issues

Several factors may impact the performance of the Data Model and Database Access Layer.

Database query efficiency can affect response times, particularly when retrieving large sets of listing or storefront data. Proper indexing of frequently queried fields, such as listing IDs or storefront IDs, can improve query performance.

Network latency between the Elastic Beanstalk application server and the AWS RDS database instance may also impact response times during database operations.

The checkout process requires multiple sequential database calls including creating the order record, inserting order items, decrementing size inventory, clearing the cart, and removing purchased items from the wishlist. Each of these operations requires a separate database interaction which could affect response times under load.

Additionally, inefficient queries or repeated database calls may increase processing time and resource usage, potentially affecting overall system performance.

3.6.3.6 Design constraints

The design of the Data Model Layer is constrained by several system-level considerations.

The application must operate with a PostgreSQL relational database system, requiring SQL-based data access and adherence to relational schema design principles.

Database communication must be compatible with the Python database adapter used by the application (such as psycopg2), which defines how queries are executed and how results are returned.

The database schema must be initialized via `init_db.py` before the application can function. Schema migrations for new columns are handled through ALTER TABLE statements executed on every application startup to ensure compatibility across environments.

The model layer must also maintain compatibility with the surrounding application architecture, including the controller and service layers that depend on structured database results for application processing.

3.7 Software Interface Description (Ryan Grimes)

The software's interface(s) to the outside world are described.

3.7.1 External machine interfaces (Ryan Grimes)

Interfaces to other machines (computers or devices) are described.

- Psycopg2 communicates to AWS RDS over port 5432. This connection is initialized in `init_db.py` and tested in `db.py`.

3.7.2 External system interfaces (Ryan Grimes)

Interfaces to other systems, products, or networks are described.

- AWS RDS: The Vault uses AWS RDS as a global database. The backend uses the psycopg2 adapter to open a secure communication channel over the TCP/IP network on port 5432 to the AWS PostgreSQL instance. This connection is protected by an AWS Security Group that only allows communications from explicitly approved IP addresses.
- AWS Elastic Beanstalk: This service automatically handles the deployment details, environment configuration, capacity provisioning, and load balancing. It is used by The Vault as the primary hosting interface for the Flask application. The application communicates with Elastic Beanstalk to manage HTTPS requests and ensure web server operability.
- AWS IAM (Identity and Access Management): This controls access permissions among the cloud infrastructure. It manages the keys used by the database admin and defines the policies that allow the Flask backend to securely upload images to Amazon S3 or query the database.
- Amazon VPC (Virtual and Private Cloud): This provides an isolated network that separates the application's backend server from the database layer. It is used to ensure that all internal traffic, especially sensitive database queries, occur within a private connection isolated from the internet.
- AWS Security Groups: These act as a virtual firewall that manages inbound and outbound network traffic. In The Vault, it is used to restrict database access to only the application server via port 5432.
- Internet Browser: The server interfaces with the browser using JSON for data and HTML/CSS for visuals.

3.7.3 Human interface (Ryan Grimes)

An overview of any human interfaces to be designed for the software is presented.

- Web GUI: A webpage at <https://127.0.0.1:5000> will contain the webpage frontend.
- Admin CLI: `init_db.py` allows for direct database schema management.

4.0 User interface design

A description of the user interface design of the software is presented.

4.1 Description of the user interface (Ryan Grimes)

A detailed description of the user interface including screen images or prototype is presented.

The program focuses on a vault/bank theme, terminology used is tech and heist related, and the primary colors are black and gold.

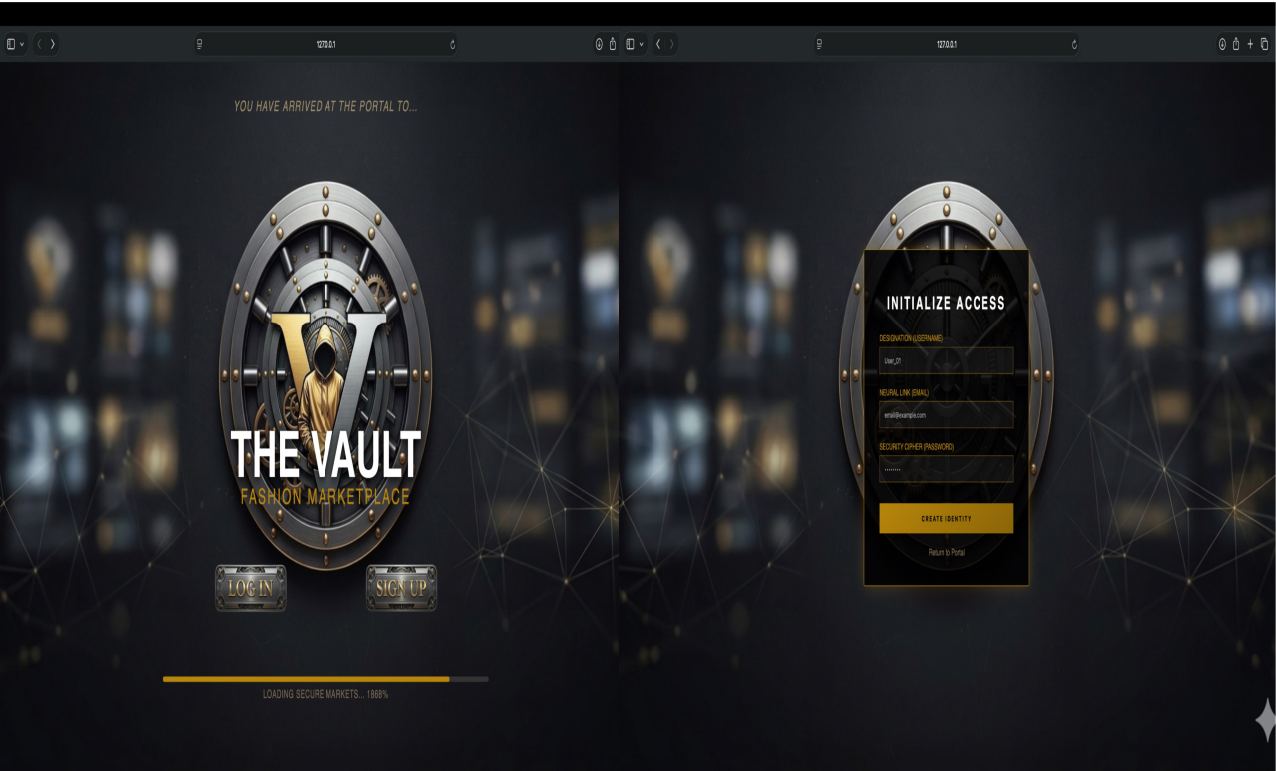


Figure 1: Default Screen

Figure 2: Signup Screen

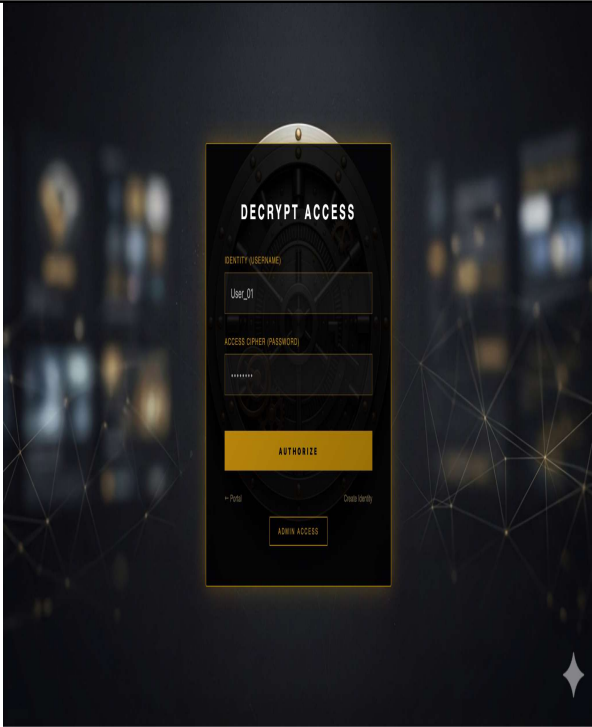


Figure 3: Login Screen

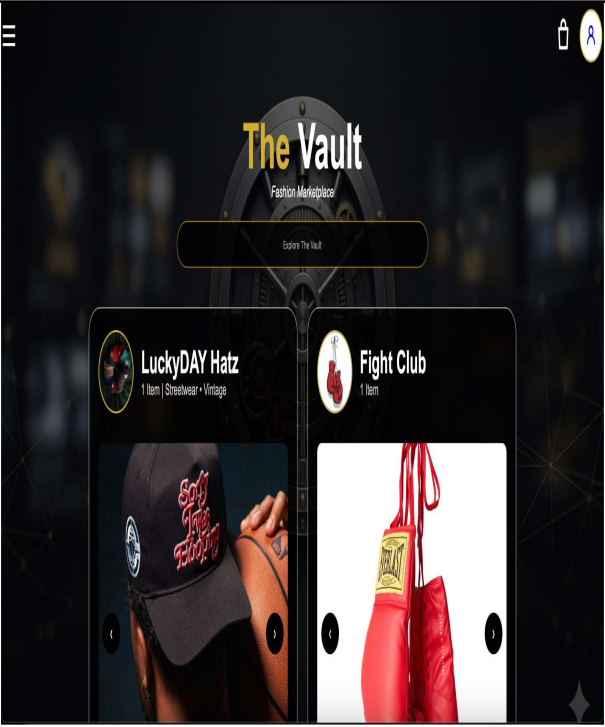


Figure 4: Storefront Homepage (Main)

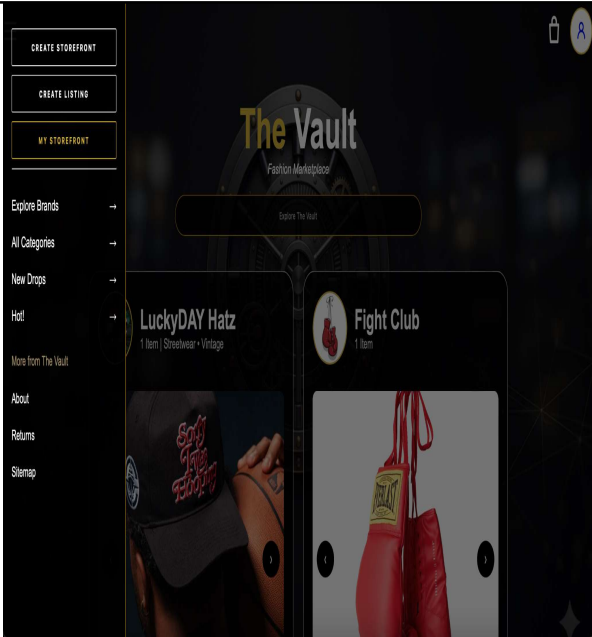


Figure 4.1: Storefront Homepage Navigation Tab

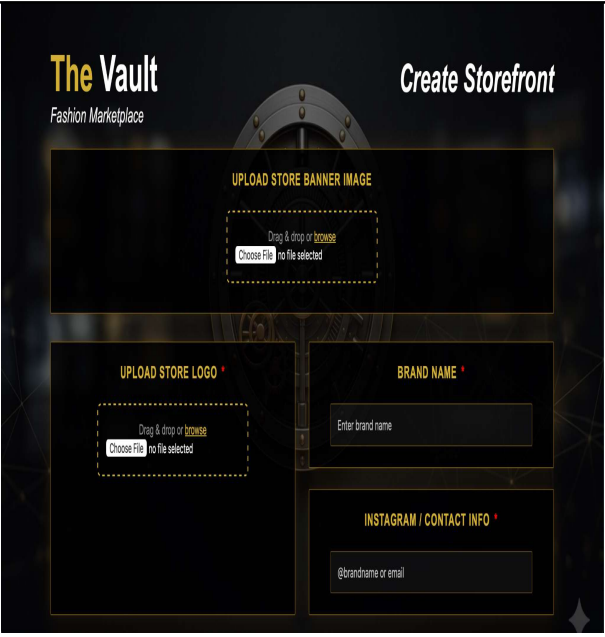


Figure 5: Create Storefront Screen

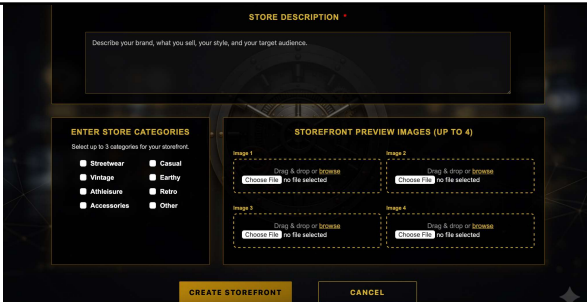


Figure 5.1: Create Storefront Screen

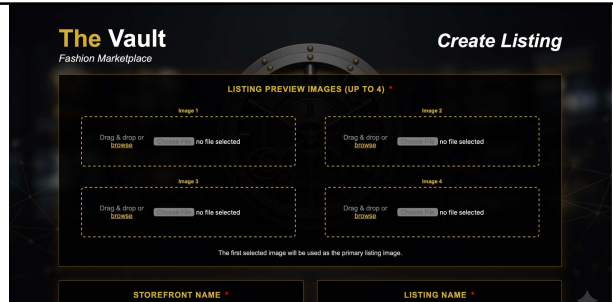


Figure 6: Create Listing Screen

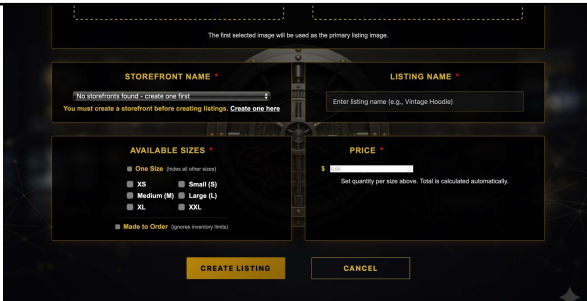


Figure 6.1: Create Listing Screen

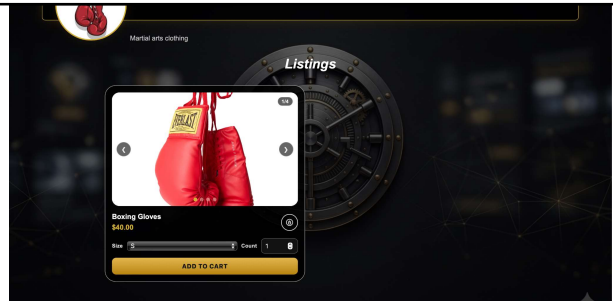


Figure 7: Listing Screen

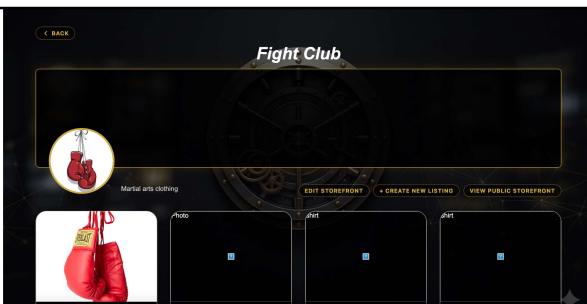


Figure 8: My Storefront Screen

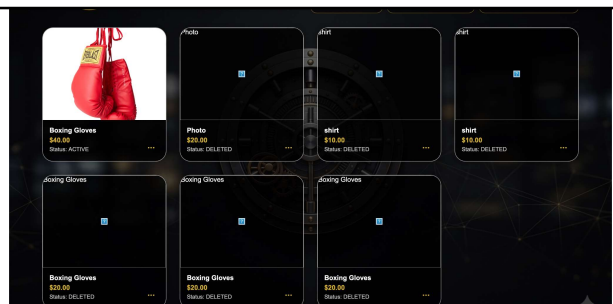


Figure 8.1: My Storefront Screen

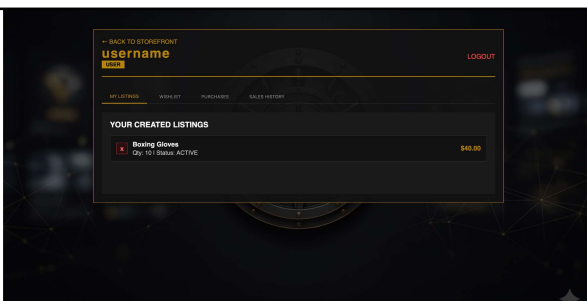


Figure 9: Account Screen (My Listings)

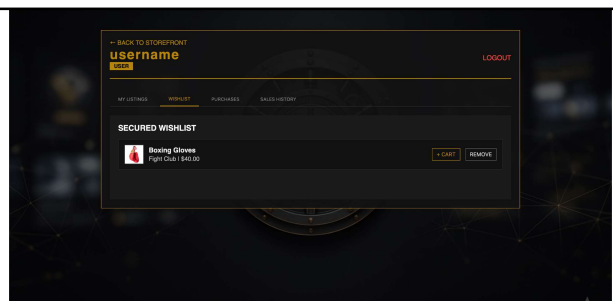


Figure 9.1: Account Screen (Wishlist)

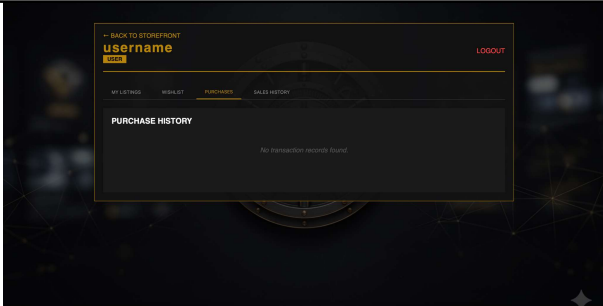


Figure 9.2: Account Screen (Purchases)

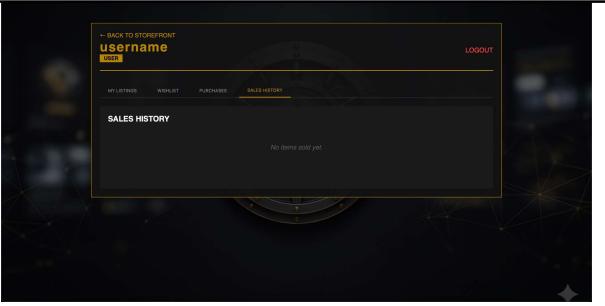


Figure 9.3: Account Screen (Sales History)

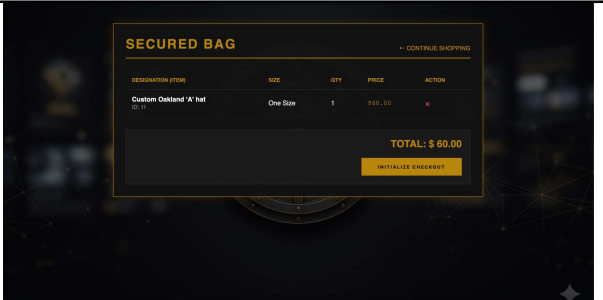


Figure 10: Cart Screen

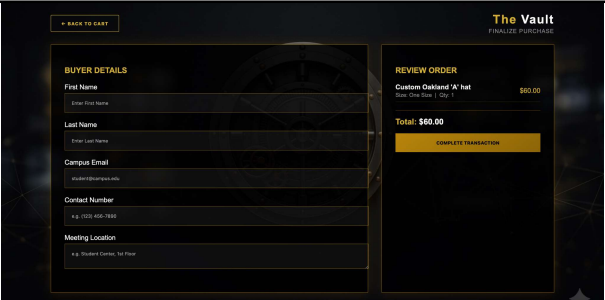


Figure 11: Checkout Screen (My Listings)

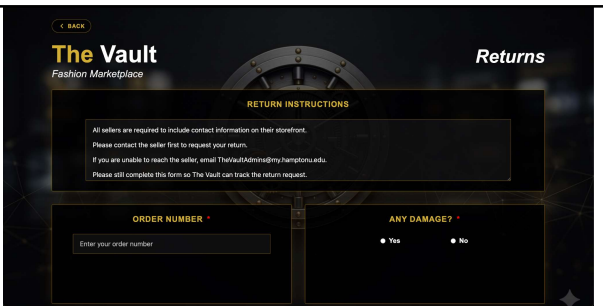


Figure 12: Returns Screen

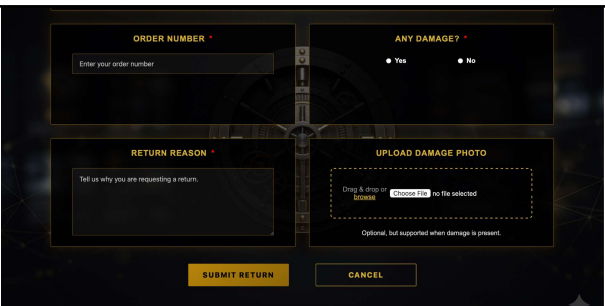


Figure 12.1: Returns Screen

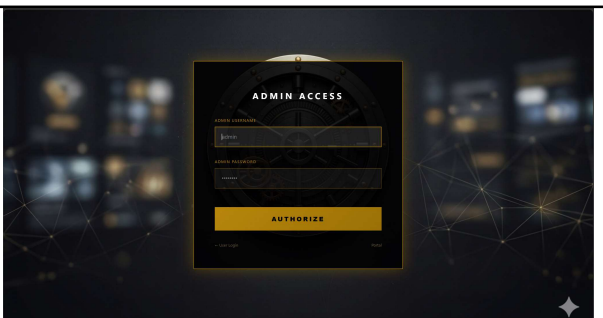


Figure 13: Admin Login Screen

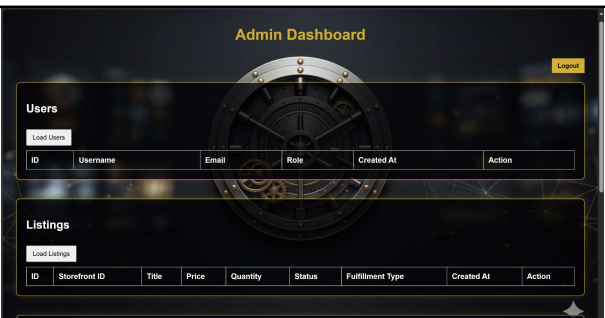


Figure 14: Admin Dashboard

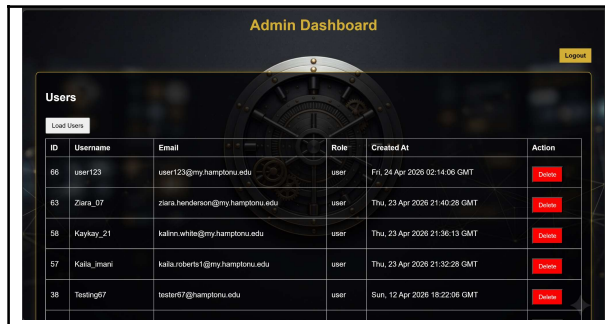


Figure 14.1: Admin Dashboard



Figure 14.2: Admin Dashboard

4.1.2 Objects and actions (Ryan Grimes)

All screen objects and actions are identified.

<u>Object</u>	<u>Action</u>
Signup Button	Checks hampton email criteria and runs register_user function
Login Button	Runs validate_login function
Enter Storefront Button	Routes user to individual storefront view at /storefronts/<storefront_id>
Wishlist Button	Adds listing to the user's wishlist, displays new card in the account page (Wishlist)
Create Storefront Button	Routes user to Create Storefront page at /storefronts/create
Edit Storefront	Allows brand owners to update their brand identity by running an UPDATE command in the RDS database
Deactivate Storefront	A button that allows users to temporarily hide their shop by toggling the is_active variable in the storefronts table
Deactivate/Reactivate Listing	A control feature for individual items to manage inventory visibility by toggling the status of a listing_id to "Hidden" or "Active"
Returns Page	A screen outlining the marketplace's refund and exchange policies
Order Confirmation Page	The post-purchase summary screen displaying item, price, date, and a unique purchase ID to the buyer
My Storefront Dashboard	A panel for sellers to view their stats and listings by running the SQL query "Select All" where the owner_id meets the current

	session.
Add to Bag Button	Adds selected listing to user's shopping bag
Menu Button	Opens side navigation pane
Bag Icon	Routes user to cart page
Profile Icon	Routes user to account page
Search Bar	Accepts user input to search/filter storefronts
Previous/Next Button(s)	Navigates between storefront listing pages
Upload Store Banner Image	Accepts image file input for storefront banner
Upload Store Logo	Accepts image file input for storefront logo
Brand Name Field	Accepts text input for storefront name
Instagram/Contact Info Field	Accepts text input for contact information
Store Description Field	Accepts text input for store description
Store Categories Checkboxes	Allows user to select one of more categories
Storefront Preview Images	Accepts up to 4 image files for listing previews
Create Storefront Button	Submits form, creates storefront object, displays new card on homepage
Cancel Button	Discards form and returns user to Storefront Homepage
Create Listing Button	Routes to create listing page
Storefront Name Field	Allows user to select which "user-owned" storefronts the listing will belong to
Listing Name Field	Accepts text input for listing name
Listing Image Field	Accepts 1 image for the listing image
Available Sizes Field	Allows user to select up to 3 sizes that are available for the listing
Count Field	Allows text input for amount available
Price Field	Allows text input for cost of the listing
Create Listing Button	Submits form, creates listing object, displays new card in the account page (My Listing)
Cancel Button	Discards form and returns user to Storefront Homepage
My Listing Button	Routes to user's listing page, displays all of the user's created listings
Wishlist Button	Routes to user's wishlist page, displays all of the user's wishlisted listings
Purchases Button	Routes to user's purchases page, displays all of the user's purchased listings
Sales History Button	Routes to user's sold history page, displays all of the user's sold listings
Back to Storefront Button	Returns user to the Storefront Homepage
Logout Button	Logs user out of their account, returns user back to the login page

4.2 Interface design rules (Ryan Grimes)

Conventions and standards used for designing/implementing the user interface are stated.

- Error transparency: Any backend error should cause a frontend error message that is easy to understand.

4.3 Components available (Ryan Grimes)

GUI components available for implementation are noted.

- Bootstrap/Tailwind CSS: provides pre-designed buttons, navigation bars, and grids that automatically shrink or grow depending on the screen size. We don't have this implemented, but may decide to do so in the future.
- Flask-WTF: When the web page is accessed, the server gives the browser a hidden, one-time-use CSRF token. The token acts as a backup factor for enforcing roles and permissions. We don't have this implemented, but may decide to do so in the future.
- FontAwesome: Provides icons and images. We don't have this implemented, but may decide to do so in the future.

4.4 UIDS description (Ryan Grimes)

The user interface development system consists of:

- VS Code Studio: Primary IDE for frontend and backend development.

5.0 Restrictions, limitations, and constraints (Madison Boyd)

Restrictions

- **Hampton University-Only Access:** The Vault is restricted to use by Hampton University students only. A Hampton University email address is required to sign in and/or create a new account. The system will validate if it is a Hampton University email account based on the ending of the email address being (hamptonu.edu).
- **Role-Based Access Control:** Users are restricted based on assigned roles. Brand owners and shoppers cannot access administrative controls and only administrators can manage all users and listings. This restriction enforces security and ownership boundaries in the system.

- **AWS Free Tier Deployment Requirement:** The system must be deployed using AWS Free Tier services. This restricted available computing resources, database sizes and storage limits, and performance abilities.

Limitations

- **AWS RDS PostgreSQL Query Editing Limitation:** When Postgres is implemented through RDS AWS the free tier does not allow for the query to be edited directly through AWS. To circumvent this, we made an `init_db.py` in VS Code that can initialize the tables in Postgres without having to upgrade the AWS plan
- **Scalability Limitation Under Free Tier:** Because the project has to be operated within the AWS Free Tier, the database storage, memory allocation, and connections are limited. Performance under high traffic or large numbers of users may be limited.
- **Development Timeline Limitation:** The project is limited by the semester timeline. Some advanced features may not be fully implemented before the project is completed.

Constraints

- **Technical Constraints:** The system must be compatible with AWS Hosting services, specifically AWS RDS for database management. The application must support cross-platform browser compatibility/access like macOS and Windows. PostgreSQL is the required relational database system for efficient storage.
- **Security Constraints:** Passwords must be stored using secure hashing methods. Sensitive payment information cannot be stored in the database system. Listing ownership must be enforced at database and application levels.
- **Resource Constraints:** The team must work within the AWS Free tier system. Development must also be completed within the project iteration timelines.

6.0 Testing Issues

Test strategy and preliminary test case specification are presented in this section.

6.1 Classes of tests (David Jackson)

- **Security Testing:** Ensures compliance with data protection policies and safeguards against vulnerabilities such as injection attacks or data leakage. Testing must also include penetration testing, and input validation checks.
- **Functional Testing:** Verifies that all specified functionalities operate as expected. Edge cases must also be tested.
- **Usability Testing:** Confirms that the system offers a clear and accessible interface.
- **Scalability and Stress Testing:** Verifies the system's ability to handle large volumes of data and spikes of traffic without significant performance degradation.
- **White-Box Testing:** All functions will be tested from the backend. Auth_survices should trigger 401 or 200 errors, and postgres commands will validate query entries and hashes.

- **Black-Box Testing:** All functions will be tested from the frontend. A user tries to signup, login, create a listing, edit a listing, delete a listing, purchase something, sell something, return something, and logout.

6.2 Expected software response (David Jackson)

- **Scalability:** Ability to process high volumes of listings and users.
- **Security Compliance:** No unauthorized access to restricted data or features, encrypted storage of sensitive data.
- **Usability:** Clear and accessible interface, ensuring seamless user interactions.
- **Fault Tolerance:** The system must be able to remain stable during unexpected failures, with automatic recovery and fallback mechanisms.

6.3 Performance bounds (David Jackson)

- **Admin Security:** Only authorized users should be able to access that admin page.

6.4 Identification of critical components (David Jackson)

Critical areas of focus while testing:

- Database connectivity
- Login authentication
- S3 connectivity
- Route functionality

7.0 Revision Logs (Madison Boyd)

Revision	By	Date	Description
1.0	The Vault	2/11/2026	The initial document was created, and an outline and table of contents were added. Basic design layouts of product and overview were added.
2.0	The Vault	2/12/26	Added content to sections 1.0-6.0
3.0	The Vault	3/5/26	Added descriptions of the user interface

			(section 4.1)
4.0	The Vault	3/21/26	Added screen images of the front end in section 4.1.1 , and added objects and actions for our application in section 4.1.2
5.0	The Vault	4/9/26 - 4/23/2026	added content to sections 2.4, 3.7.2, 4.1.2, 6.4. Updated 3.1, 3.2.2, 3.2.3, 3.3.3.5-3.3.3.6, 3.4.1-3.4.3.6, 3.5-3.5.3.6, 3.6.2-3.6.3.6, 4.1.2 to reflect the fully completed and deployed product.